



University
of Glasgow

DoW DoM M 2020

xx.xx - yy.yy

(90 minutes)

DEGREES of MSci, MEng, BEng, BSc, MA and MA (Social Sciences)

BIG DATA H

Answer all questions

This examination paper is worth a total of 60 marks.

The use of calculators is not permitted in this examination.

INSTRUCTIONS TO INVIGILATORS: Please collect all exam question papers and exam answer scripts and retain for school to collect. Candidates must not remove exam question papers.

1. (a) A HDFS client reading data from a file on HDFS:
- a. Receives all file data through the NameNode
 - b. Receives the location of file blocks from the NameNode
 - c. Receives both the data and block locations from the NameNode
 - d. Receives both the data and block locations from a DataNode

Choose the single correct answer and explain the effect of this design on the performance and scalability of the system, as well as the requirements it imposes on the affected components [5 points]. Also discuss why non-selected options are wrong. [3 points].

[8]

Solution: (Bookwork for the MCQ part (not graded); Synthesis for the rest)

Correct answer: (b).

Clients will contact the NameNode only to retrieve the locations of file blocks, and also cache them to avoid repeated queries to the NameNode; all subsequent I/O is then performed directly with the respective DataNodes. This allows the system to scale exceptionally well despite the presence of a single point of contact (NameNode), as the latter is explicitly removed from the “critical path” of data transfers. [3 points].

However, the assumption is that the NameNode will be able to maintain a very low latency in its responses to client requests. This in turn requires that the file system metadata can all fit in the NameNode’s RAM. This then poses either extra requirements on the specs of the NameNode host or an upper limit to the size of the file system (number of files, number of blocks, etc.) [2 points].

Option (a) and (c) would render the NameNode a bottleneck in the system and would remove any gains from the distributed nature of the system. [1 point].

Option (d) would require that all DataNodes know the location of all blocks; this would put an inordinate amount of load on the network as any block creation/migration would need to be broadcast to all DNs. [1 point] It would also mean that DNs would have the same high memory requirements as the NN, as explained above. [1 point].

(Full/partial marks will be awarded on the basis of correctness and completeness of arguments made)

- (b) Considering the functionality and operation of NameNodes and DataNodes, one would expect:
- a. The RAM and disk capacity of all machines (i.e., both those hosting a NameNode and those hosting a DataNode) to be of the same (commodity) grade
 - b. The disk capacity of machines hosting NameNodes to be higher than of machines hosting DataNodes
 - c. The RAM capacity of machines hosting NameNodes to be higher than of machines hosting DataNodes
 - d. The disk and RAM capacity of machines hosting DataNodes to be higher than of machines hosting NameNodes

Choose the single correct answer and explain the reasoning behind your choice.

[6]

Solution: (Synthesis)

Correct answer: (c) [0 points].

NameNodes need to be able to store the complete filesystem image in their RAM. [1 point]. Disk capacity is not that important for NameNodes (within reasonable bounds) as one can expect a typical disk to be (much) larger than a machine's RAM. [2 points]. DataNodes, on the other hand, need only moderate RAM for caching of blocks [1 point] but require a higher disk capacity [1 point] as they are where the bulk of the data is stored. [1 point].

Full marks will also be awarded if the latter part is answered via reduction ad absurdum.

2. (a) You are given a dataset in text format where each line contains two integer values (as strings) separated by space, the first denoting an item's weight and the second denoting its volume. We want to design a MapReduce job that will return all distinct combinations of these two values that exist in the dataset; in other words, it will be implementing the statement "SELECT DISTINCT Weight, Volume FROM Dataset". Provide the pseudocode for the `map(key, value)` and `reduce(key, value[])` functions (alternatively, explain in plain English what these functions will do), assuming that the default text input format is used.

[5]

Solution: (Problem solving)

MapReduce will group map output keys before feeding them to the reducers. We can thus make use of this functionality to implement the `DISTINCT` operator. For each input pair of numbers, the mapper can transform the two numbers into (32-bit) integers and emit a key-value pair with the key set to the binary concatenation of the two integers and the value set to null [3 points]. The reducer would then merely output the key (possibly in string representation), regardless of the value(s) associated with it [2 points].

Alternatively, the mapper could simply emit a key-value pair with the input value (`<number 1>`, `<number 2>`) as the key and null as the value. This solution would be awarded 2 points for the mapper as it produces the correct result but results in higher storage/network overhead than the earlier solution.

Other solutions will be considered on the basis of their correctness and efficiency.

- (b) Extend your solution to also use a combiner; i.e., provide the pseudocode for its `reduce(key, value[])` function (or explain in plain English what this function will do). Also discuss any amendments you may need to perform to your `map(...)` and/or `reduce(...)` functions so that they work correctly when this combiner is defined.

[3]

Solution: (Problem solving)

The combiner would act as a mini reducer in this case. Specifically, it would output a single key-value pair with the key set to the input key and the value being a single null (i.e., would “combine” all copies of a given weight, volume combination into a single one) [2 points]. No amendments are required to the other two functions in this case [1 point].

Other solutions will be considered on the basis of their correctness and efficiency.

- (c) Is it possible for reducers to start executing before all mappers are finished? Also, is it possible for reducers to start producing output while some mappers are still executing? What would be the performance repercussions in each case? Explain your answer.

[3]

Solution: (Bookwork and Synthesis)

As each reducer will need to merge the output of multiple (possibly all) mappers; hence, it is necessary to wait for all mappers to finish before reducers can start their processing and production of output. [1 point]. Reducers may opt to start *transferring* data before all mappers finish (also known as the slowstart mechanism in MR lingo). [1 point]. This results in network utilisation going up but this is for transfers that would happen anyway; letting this process start when a good percentage of the mappers has finished, further means that we can interleave the CPU-intensive processing performed by mappers with the IO-intensive task of the fetch phase of MR’s shuffle. [1 point].

- (d) Spark optimises its processing on the RDD graph by breaking it down into stages, based on the dependencies among RDDs. Given a parent and a child RDD, the dependency between them is:
- Narrow when each partition of the parent RDD is used by at most one partition of the child RDD and wide when each partition of the parent RDD is used by multiple child RDD partitions
 - Narrow when each partition of the parent RDD is used by multiple child RDD partitions and wide when each partition of the parent RDD is used by at most one partition of the child RDD.
 - Ignored if all partitions of one or the other RDD reside on a single host.
 - Considered only for expensive operations such as joins and group-by’s.

Choose the single correct answer, then discuss how these dependencies are used to create the stages [3 points] and explain how and why this design makes sense from a performance/efficiency point of view [6 points].

[9]

Solution: (Bookwork for the MCQ part and the use of dependencies in the creation of stages; Synthesis for the explanation)

Correct answer: (a) [0 points]

Narrow operations are grouped and pipelined in a stage. That is, tasks on RDD partitions with narrow dependencies are scheduled on the executor holding the parent partition. Wide operations require data to be shuffled around the computing nodes and as such signify the boundaries of stages. [3 points]

This design tries to replicate what a seasoned MapReduce programmer would do by hand. That is, identify operations that can be carried out based solely on data local to each executor, and execute it in a pipelined fashion. In other words, if all it takes to output a key-value in a given partition of RDD Z, is computation on a single key-value from a given partition of RDD Y, which in turn requires computation on a single key-value from a given partition of RDD X, then it makes sense to group said computations so that they are applied in memory one after the other (i.e., grouped and pipelined) to the data of RDD X; as an executor goes through the key-value pairs of a partition of RDD X, it can perform both computations (to produce Y and Z) in one go. [5 points]. This results in a significant reduction in network overhead, for negligible memory overhead at the executors (in the general case). [1 point]

(Full/partial marks will be awarded on the basis of correctness and completeness of arguments made)

3. (a) Consider a Log-Structured Merge Tree (LSM)-based data store. Indicate whether each of the following statements is true or false and briefly explain your answer ([2 points each]):
- The data store is generally expected to exhibit lower read latency as more items are added.
 - The data store is generally expected to exhibit high write throughput.
 - Each compaction results in less items being retrievable from the data store.
 - LSM-based data stores can only be implemented using a master-workers architecture.

[8]

Solution: (Bookwork for the MCQ part; Synthesis otherwise)

Correct answers:

- False. In the general case, the read latency of an LSM tree-based data store is expected to increase as items are added, due to the increase in the number of spill files that need to be scanned in order to produce the result set. Caching may mitigate this overhead, especially for query workloads exhibiting a highly skewed access pattern, but in the general case this wouldn't be enough to offset the impact of the former.
- True. The combination of in-memory buffering (data first stored in a buffer in RAM and spilled into an immutable sorted file on disk once full) and write-ahead

- logging (append-only operation, possibly buffered and amortised over multiple insertions), is expected to allow for a high write throughput in the general case.
- c. False (in the general case). Compactions are only expected to remove non-visible data items; e.g., items "hidden" by newer versions or tombstone records. As such, a compaction won't affect the number of items retrievable from the data store. An exception to this rule would be data stores allowing for varying numbers of versions to be returned to each query, but one could argue that the versions missed in this case should not be expected to normally be part of the result set.
 - d. False. The LSM design applies on a per-storage-node basis and as such it dictates nothing with regards to the overall architecture. As a matter of fact, both HBase/BigTable (master-workers architecture) and DynamoDB/Cassandra (peer-to-peer architecture) implement an LSM-based model.

(Answers will be marked on the basis of the provided explanation; that is, if an answer gives the wrong True/False indication but provides correct arguments, it won't be penalised – hence why the True/False part hasn't been given a separate mark).

(b) Would you consider BigTable/HBase row stores or column stores? Explain your answer.

[4]

Solution: (Synthesis)

BigTable/HBase are essentially row stores, although often referred to as "fat column stores". Cells are sorted first by rowkey, then by column identifier, resulting in a row-major storage format. They do borrow ideas from column stores, in the sense that columns are grouped together in column families and each column family is stored in separate files/directories – this means that if one was to query data in only one column family, they wouldn't need to touch data in other column families. Yet, within each such column family, the data would be still stored in a row-major format. For them to be considered column stores, it would require that either each column is in a column family of its own, or that all queries only access one column family each.

(Full/partial marks will be awarded on the basis of correctness and completeness of arguments made)

(c) Consider a distributed data store which uses Consistent Hashing with an order-preserving hash function to assign data items to storage nodes. Indicate whether the following statements are true or false and explain your answer, giving examples if necessary [2 points each].

- a. An item would be hashed to the same storage node regardless of fluctuations in the number of nodes in the system.
- b. The storage load would generally be balanced across nodes.
- c. Range queries are expected to perform well.
- d. Point (equality) queries are expected to perform better than with a random hash

function.

[8]

Solution: (Bookwork plus Synthesis)

- a. False. It depends on the IDs of nodes added/removed to/from the system. Consider two storage nodes with IDs A and B , and a data item with ID x , so that $A < x < B$. Assume that data items are stored at the successor of their ID. Then, x would be stored at node B . If nodes are added/removed outside the interval $[A, B]$ then x 's placement wouldn't change. However, if node B is removed or if another node C is added so that $x < C < B$, then x 's placement would change.
- b. False (in the general case). As the hash function is order preserving, the distribution of storage load across nodes would follow the distribution of item values. We can generally assume that the IDs of storage nodes are distributed evenly (quasi-uniformly) around the ID space. Then, the storage load will be balanced only if the distribution of item values is uniform (not that likely), and imbalanced otherwise.
- c. True. This is the main premise of using an order-preserving hash function in the context of Consistent Hashing.
- d. False. Point queries need to access only one storage node each; as such, using an order-preserving hash function won't affect their performance positively. As a matter of fact, if anything, we might expect an order-preserving hash function to have a negative impact on the basis of load imbalances created, as discussed in the answer to (b).

(Full/partial marks will be awarded on the basis of correctness and completeness of arguments made)

- (d) Consider a distributed data store where each item has R replicas (i.e., R instances of the item stored on nodes across the system). Let r be the number of replicas that must be contacted when reading an item, and w be the number of replicas that must be contacted when writing an item. Which of the following must hold in order for the system to provide strong consistency guarantees (linearizability) in the face of errors during reads and writes when using quorum-based replication? Discuss any assumptions and explain your answer.
- a. Set r and w so that $r + w > N$ and $w + w > N$.
 - b. Set r and w so that either $r \geq N/2$ or $w \geq N/2$.
 - c. Set r and w so that $r + w > N$ and $r > w$.
 - d. None of the above.

[6]

Solution: (Synthesis)

One may think that (a) is the correct answer as that is what is described as “strong consistency” in the Cassandra paper. However, the underlying assumption is that writes

always succeed; that is, that w replicas have been created correctly and fully, before a subsequent read of r replicas is initiated. However, for that to be of any import one first needs to guarantee that, given an update to an item i , either all w replicas are updated, or otherwise all updates are rolled back. This requires a much stronger/stricter protocol than simple quorum (e.g., undo/redo logs plus 2PC, Paxos, Raft, etc.). In the absence of such mechanisms, linearizability cannot be guaranteed and hence the correct answer is (d).

(Full/partial marks will be awarded on the basis of correctness and completeness of arguments made)